Project Report on

AI Based Meeting-Bot: A Cognitive Intelligence Solution for Enterprise Meetings using Sentiment Analysis & Hybrid RAG Strategy

Submitted By

Subhadeep Mondal
Registration No.: D01-1112-0185-22
Roll No.: T91/IT/224148

Syed Tarifuddin Ahmed
Registration No.: D01-1115-0149-22
Roll No.: T91/IT/224151

Semester: 7th Semester

Stream: B.Tech, Information Technology

Mini Project (IT-796)

Under the guidance of,

Prof. Soumya Sen
A.K.Choudhury School of Information Technology

TABLE OF CONTENTS

Section	Heading / Topic	Page No.
1.0	Preface	3
2.0	Introduction	4
3.0	Problem Definition & Significance	5
3.1	Problem Definition: *The Crisis of "Corporate Amnesia"*	5
3.2	Critical Analysis of Existing Works	6
3.3	Significance	7
4.0	Purpose and Motivation	8
5.0	Literature Survey	9
6.0	Proposed Methodology	11
6.1	Phase 1: Ingestion *(The "Ears")*	11
6.2	Phase 2: Cognition *(The "Brain")*	12
6.3	Phase 3: Synthesis *(The "Interface")*	13
7.0	Algorithms & Flowchart	14
7.1	Algorithm 1: *Diarized Temporal Sentiment Analysis*	14
7.2	Algorithm 2: *Hybrid RAG Retrieval Strategy*	16
—	Current Prototype *(Visuals & Screenshots)*	18
8.0	Future Improvements	19
9.0	Conclusion	20
10.0	References	21

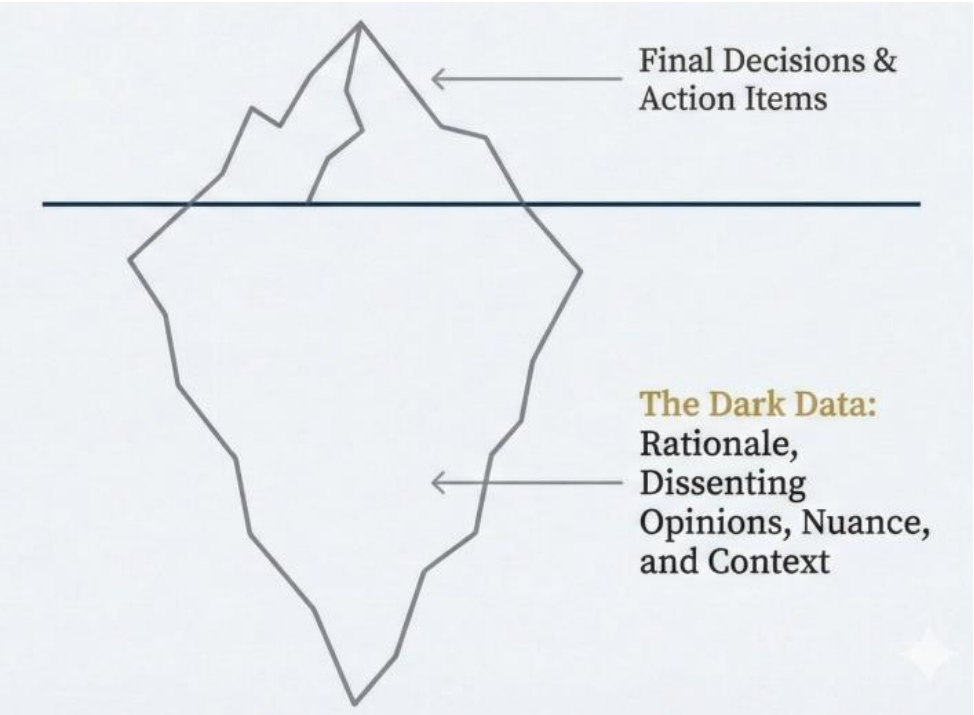
1. Preface

This report documents the design, development, and implementation of "MeetingBot," an autonomous cognitive intelligence platform designed to address the challenges of unstructured data loss in enterprise communications. The project explores the intersection of Generative AI, Knowledge Graph theory, and Event-Driven Software Architecture. This document serves as a comprehensive record of the research methodology, architectural decisions, and technical implementations undertaken to transform ephemeral voice conversations into persistent, structured business intelligence. It is intended for academic evaluation and technical review.

2. Introduction

In the contemporary digital landscape, the volume of corporate communication occurring via video conferencing (Zoom, Google Meet, Microsoft Teams) has grown exponentially. However, these interactions typically generate "dark data"—unstructured audio streams that are difficult to search, analyze, or synthesize.

MeetingBot is a SaaS (Software as a Service) platform engineered to automate the lifecycle of this data. From the moment a meeting is scheduled, the system handles autonomous capture, transcription, and cognitive processing. Unlike simple transcription tools, MeetingBot leverages a **Hybrid Retrieval-Augmented Generation (Hybrid RAG)** architecture, combining Vector Databases for semantic search with Graph Databases for structured relationship mapping. This allows the system to not only record *what* was said but to "understand" the hierarchical relationships between decisions, risks, and personnel. The platform is built on a modern, scalable stack utilizing Next.js 15, AWS Lambda, Neo4j, and Pinecone, ensuring enterprise-grade performance and security.

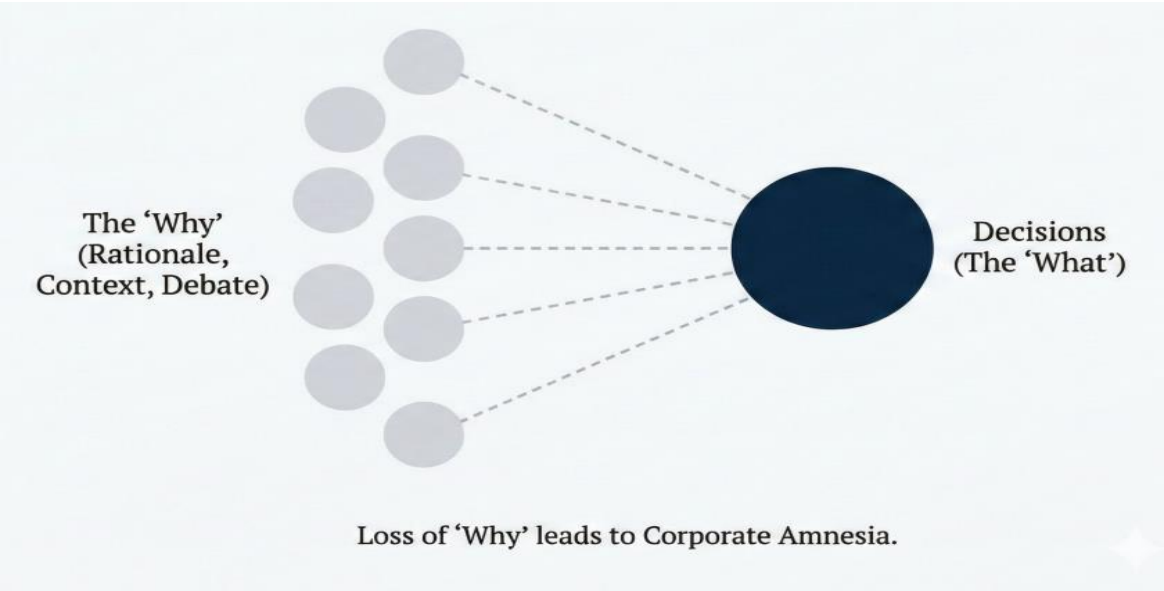


3. Problem Definition & Significance

3.1 Problem Definition: The Crisis of "Corporate Amnesia"

The primary problem addressed by this research is **"Corporate Amnesia"**—the rapid decay of critical information after a meeting concludes. In standard corporate workflows, intellectual property generated during spoken debates is rarely captured effectively.

1. **Context Loss:** Traditional meeting minutes capture the final decision but discard the *rationale* and the *dissenting arguments* (the context). When the decision environment changes, teams lack the historical data to understand why a choice was made.
2. **Information Silos:** Meeting data remains trapped in the memory of attendees. A project manager in one department has no algorithmic way to discover that an engineering team in another department discussed a dependency that impacts their timeline.
3. **Subjective Bias:** Manual note-taking is inherently biased by the scribe's perspective, leading to disputed accountability regarding action items and deadlines.



3.2 Subpoint: Critical Analysis of Existing Works

Current solutions generally fall into three categories, each presenting significant limitations:

- **Manual Documentation:**
  - *Mechanism:* A designated attendee types notes during the call.
  - *Limitation:* This is resource-intensive and prone to human error. The scribe cannot actively participate in the discussion while typing, and the resulting record often lacks technical fidelity.
- **Native Platform Recording (Zoom/Teams):**
  - *Mechanism:* The platform records raw video/audio and generates a flat transcript.
  - *Limitation:* This produces "dead data." The files are not indexed for semantic meaning. To extract a specific insight, a user must manually scrub through hours of footage. There is no cross-meeting intelligence or synthesis.
- **First-Generation AI Wrappers (e.g., Otter.ai):**
  - *Mechanism:* Uses purely text-based Large Language Models (LLMs) to summarize transcripts.
  - *Limitation:* These systems rely on **Vector-Only RAG**, which excels at thematic matching but fails at structural reasoning. They suffer from "hallucinations" (fabricating facts) and treat every meeting as an isolated event, failing to connect dots between separate conversations (e.g., detecting that a budget cut in Meeting A blocks a project in Meeting B).

3.3 Significance

The significance of MeetingBot lies in shifting the paradigm from **Passive Recording** to **Active Intelligence**. By automating the capture and structuring of meeting data, the system solves the "Accountability Gap" by creating an immutable record of assigned tasks. Furthermore, it democratizes access to information, acting as an automated "Chief of Staff" that allows any employee to query the collective oral history of the organization.

4. Purpose and Motivation

The motivation behind this project is to bridge the gap between unstructured human communication and structured database logic. The purpose is to build a system that acts as an unbiased, omnipresent observer in the workplace. By removing the cognitive load of note-taking, MeetingBot allows professionals to focus on creative problem-solving and active listening. Furthermore, the motivation extends to solving the "Groupthink" phenomenon; by engineering adversarial AI agents (the "Devil's Advocate"), the system is designed to objectively flag risks and logical fallacies that human subordinates might be too hesitant to voice to leadership.

5. Literature Survey

This research builds upon ten foundational pillars of modern AI and software engineering:

1. **"Attention Is All You Need" (Vaswani et al., 2017):**
   - *Summary:* This paper introduced the Transformer architecture, which underpins modern LLMs like GPT-4. Its self-attention mechanism allows the model to process long sequences of text (such as hour-long transcripts) in parallel, capturing long-range dependencies in conversation that previous RNN architectures missed.
2. **"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (Lewis et al., 2020):**
   - *Summary:* Proposed RAG as a solution to model hallucinations. It demonstrated that providing an LLM with external, retrieved documents (like transcript chunks) significantly improves factual accuracy compared to relying solely on the model's pre-trained memory.
3. **"GraphRAG: Unlocking LLM Discovery on Narrative Private Data" (Microsoft Research, 2024):**
   - *Summary:* This pivotal research highlighted the limitations of Vector RAG in answering "global" questions. It proposed using Knowledge Graphs to map structured relationships. This directly influenced our decision to use Neo4j to map "People-to-Project" relationships.
4. **"BERT: Pre-training of Deep Bidirectional Transformers" (Devlin et al., 2018):**
   - *Summary:* Established the standard for embedding models. We leverage modern derivatives (OpenAI text-embedding-3) to convert meeting text into high-dimensional vectors, enabling the system to understand semantic similarity beyond keyword matching.
5. **"VADER: A Parsimonious Rule-based Model for Sentiment Analysis" (Hutto & Gilbert, 2014):**
   - *Summary:* While dated, this paper defined the challenges of analyzing social text (sarcasm, negation). Our "Temporal Sentiment Arc" evolves this by adding the dimension of time and speaker diarization, rather than calculating a single static score.
6. **"Speaker Diarization with LSTM" (Google, 2017):**
   - *Summary:* Explored deep learning methods to answer "Who spoke when?" Our project relies on upstream diarization to

attribute sentiment and action items to specific speakers, which is critical for multi-agent analysis.

7. **"Serverless Architectures: Throughput and Response Time" (Various):**
    - *Summary:* Analyzes the scalability and "cold start" issues of Function-as-a-Service (FaaS). This literature informed our architectural decision to use asynchronous message queues (Upstash QStash) to handle long-running AI processes without hitting HTTP timeouts.
8. **"The Property Graph Database Model" (Neo4j Technical Papers):**
    - *Summary:* Defines the data model where entities (nodes) and relationships (edges) are first-class citizens. This structure allows for $O(1)$ traversal speeds, enabling our system to perform real-time "multi-hop" reasoning across thousands of meeting nodes.
9. **"Ethical Risks of AI in Workplace Surveillance" (Recent Ethics Papers):**
    - *Summary:* Discusses the privacy implications of recording employees. This literature motivated our "Privacy-First" design, ensuring the bot explicitly announces its presence and provides robust deletion protocols.

6. Proposed Methodology

The methodology adopts a **Cognitive Pipeline Architecture**, treating the flow of data as a series of transformations from unstructured signals to structured intelligence.

Phase 1: Ingestion (The "Ears")

The system employs an **Event-Driven Strategy**.

- **Trigger:** Integration with the Google Calendar API detects upcoming meetings.
- **Deployment:** An AWS EventBridge Scheduler triggers a bot instance (via Meeting Baas) to join the video call (Zoom/Teams/Meet) via WebRTC.
- **Capture:** The bot captures raw audio streams and performs real-time Speech-to-Text (STT) conversion, generating a diarized transcript.

Phase 2: Cognition (The "Brain")

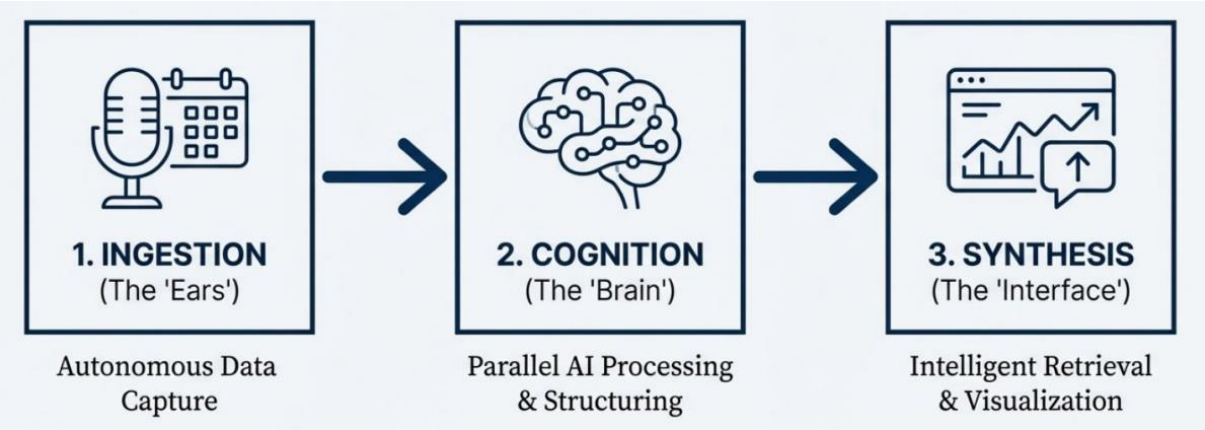
To handle high computational loads, processing is decoupled from ingestion.

- **Orchestration:** Raw transcripts are pushed to **Upstash QStash** (a serverless message queue). This ensures zero data loss even during traffic spikes.
- **Parallel Processing:** A specialized "Worker API" consumes the job and splits the task into three parallel streams:
    - **Vector Agent:** Chunks text into semantic segments and generates embeddings for the **Pinecone** vector database.
    - **Graph Agent:** An LLM extracts named entities (e.g., "Project Alpha", "Server Crash") and writes them as nodes/edges to the **Neo4j** graph database.
    - **Analyst Agent:** Performs adversarial risk analysis ("Devil's Advocate") and temporal sentiment mapping.

Phase 3: Synthesis (The "Interface")

- **Visualization:** The frontend (Next.js 15) queries the processed data. It utilizes **React Force Graph** to visualize the Knowledge Graph and **Recharts** to render the Multi-Speaker Emotional Arc.

- **Retrieval:** A chat interface allows users to query the data using the Hybrid RAG engine, synthesizing answers from both the vector and graph indices.

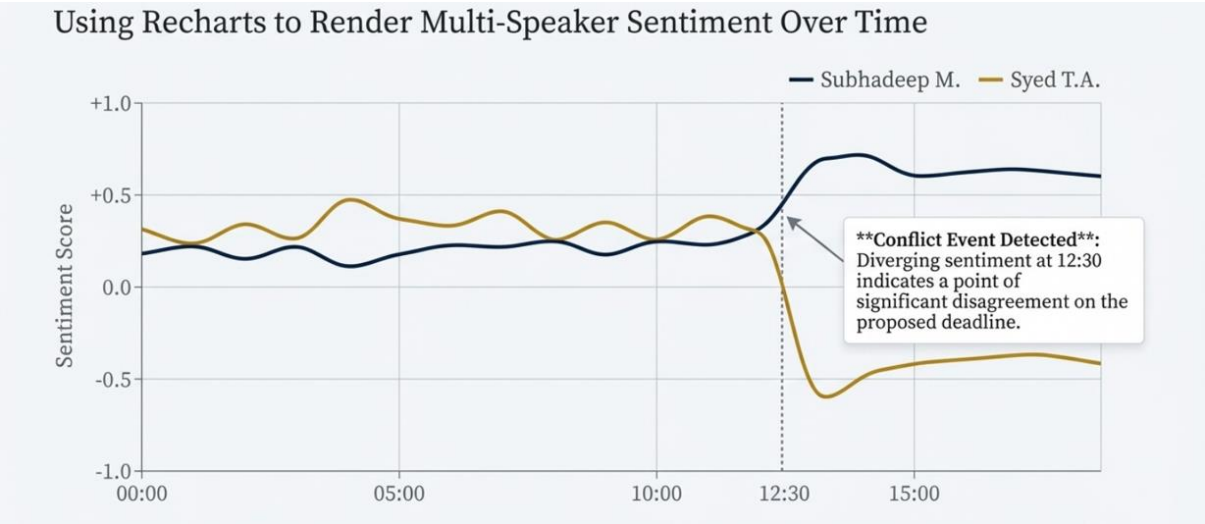


7. Algorithms & Flowchart

Algorithm 1: Diarized Temporal Sentiment Analysis

Goal: To visualize the emotional journey of *each specific speaker* throughout the meeting, identifying moments of conflict or consensus.

- **Step 1: Input Acquisition**
  - **Action:** The system receives the final transcript JSON, which contains words, timestamps, and speaker labels (e.g., Speaker A, Speaker B).
  - **Explanation:** We cannot analyze the whole text at once because emotions change minute by minute.
- **Step 2: Temporal Segmentation (Slicing)**
  - **Action:** The transcript is sliced into **30-second time windows** (e.g., 00:00–00:30, 00:30–01:00).
  - **Explanation:** A 30-second window is the "Goldilocks" zone—short enough to catch quick reactions, but long enough to capture a complete sentence.
- **Step 3: Speaker Grouping (Diarization)**
  - **Action:** Inside each 30-second window, the text is separated by speaker.
  - **Example:**
    - *Input:* [00:10] Alice: I disagree. [00:15] Bob: It's fine.
    - *Output:* { Alice: "I disagree", Bob: "It's fine" }
  - **Explanation:** This allows us to score Alice and Bob separately, rather than averaging them into a "neutral" score.
- **Step 4: AI Scoring**
  - **Action:** The grouped text is sent to the LLM (Large Language Model) with a prompt to rate each speaker on a scale from **-1.0 (Negative)** to **+1.0 (Positive)**.
  - **Explanation:** The AI understands context (e.g., sarcasm or hesitation) better than simple keyword matching.
- **Step 5: Graph Generation**
  - **Action:** The scores are plotted on a multi-line chart where the X-axis is Time and the Y-axis is Sentiment.
  - **Explanation:** Crossing lines (one going up, one going down) visually indicate a "Conflict Event" to the user.

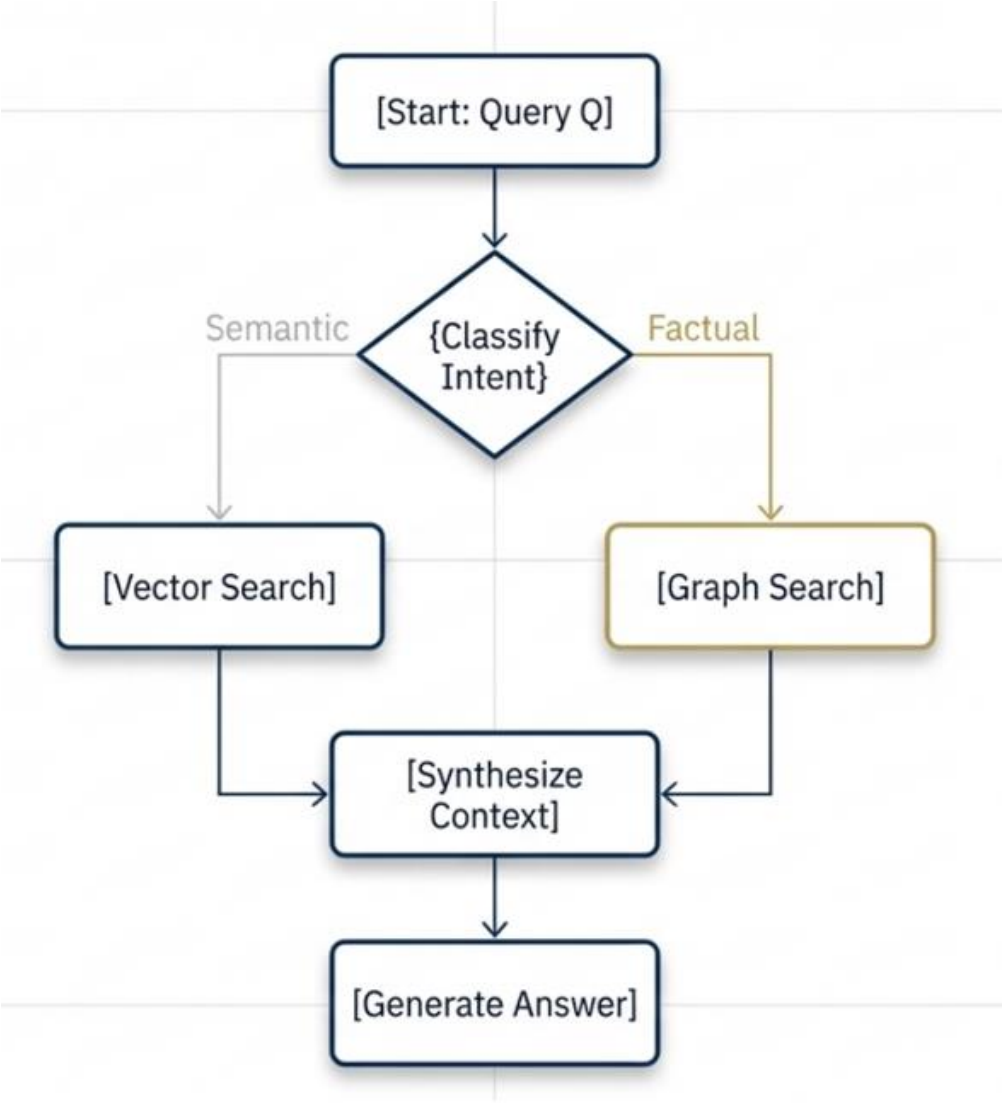


Algorithm 2: Hybrid RAG Retrieval Strategy

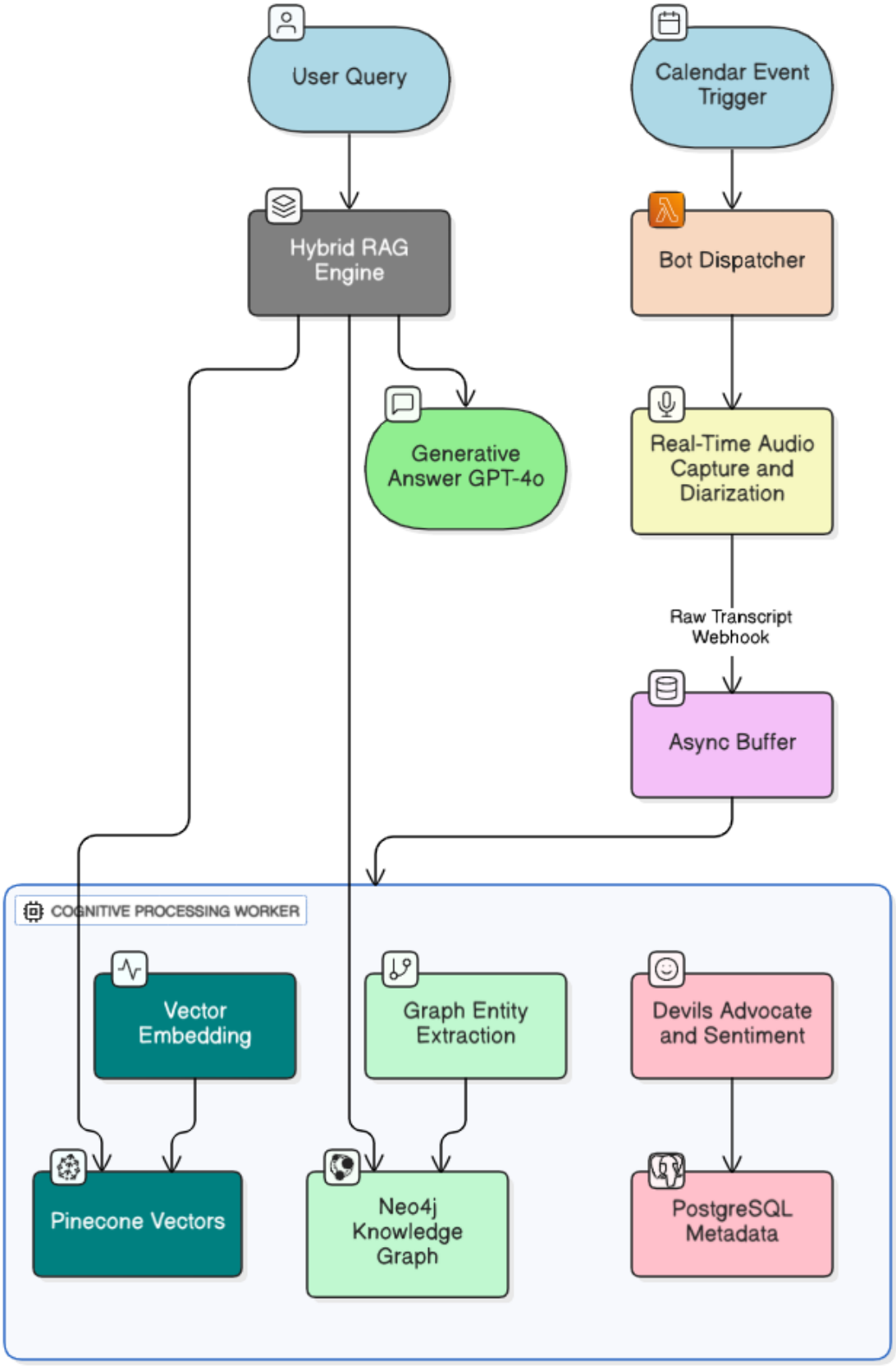
Goal: To answer user questions by combining "fuzzy" conceptual search (Vectors) with "hard" factual search (Graphs).

- **Step 1: Intent Classification (The Router)**
  - **Action:** The system analyzes the user's question to decide what kind of information is needed.
  - **Explanation:** If the user asks *"How did the team feel?"*, we need **Vectors** (Concepts). If the user asks *"Who manages Project Alpha?"*, we need **Graphs** (Facts).
- **Step 2: Parallel Retrieval (Dual-Path Search)**
  - **Path A (Vector Search):**
    - The system searches the **Pinecone Database** for transcript chunks that are semantically similar to the question.
    - *Result:* Returns the "vibe" or context of the discussion.
  - **Path B (Graph Search):**
    - The system extracts keywords (Entities) from the question and traverses the **Neo4j Knowledge Graph**.
    - *Result:* Returns strict relationships (e.g., User A -> BLOCKS -> Project B).
- **Step 3: Context Synthesis**
  - **Action:** The system combines the results from Path A (Context) and Path B (Facts) into a single block of text.
  - **Explanation:** This gives the AI the complete picture—both the unstructured conversation and the structured facts.
- **Step 4: Answer Generation**

- **Action:** The synthesized context is fed into GPT-4o with instructions to answer the user's question using *only* the provided information.
- **Explanation:** This restriction prevents "hallucinations" (inventing facts), ensuring the answer is grounded in reality.



FLOWCHART


User Query
Calendar Event Trigger
Hybrid RAG Engine
Bot Dispatcher
Generative Answer GPT-4o
Real-Time Audio Capture and Diarization
Raw Transcript Webhook
Async Buffer
COGNITIVE PROCESSING WORKER
Vector Embedding
Graph Entity Extraction
Devils Advocate and Sentiment
Pinecone Vectors
Neo4j Knowledge Graph
PostgreSQL Metadata

SAMPLE INPUT & OUTPUT

Input: Transcript $T = \{(w_i, t_i, s_i)\}$

Transformation: $F(T) \rightarrow KB = (V, G)$

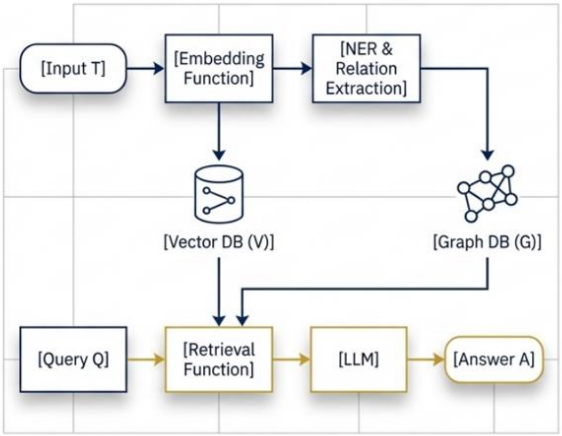
V: Vector space of semantic embeddings.

G: Knowledge graph $G = (E, R)$ of entities and relations.

Objective: $\max P(A \mid Q, KB)$

Maximize probability of correct Answer A given Query Q and Knowledge Base KB.

Data → Embedding → Inference Flow



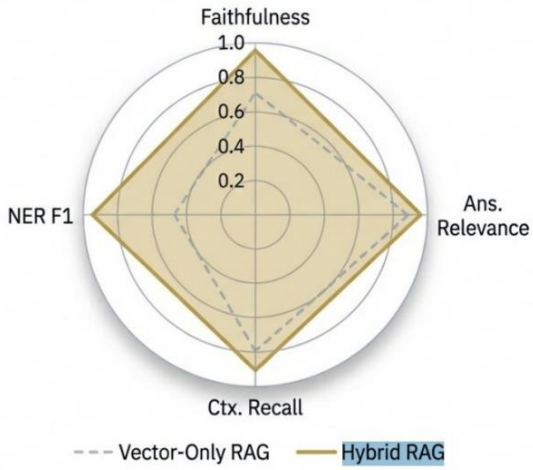
- **Query:** "Who is assigned to fix the server crash blocking Project Alpha?"
- **Vector-Only Result:** "Discussion of... "Project Alpha" being blocked... mentions "server crash"... Alice talked about fixing things." (Ambiguous)
- **Hybrid RAG Result:** Retrieves graph path: `(Entity: Alice)-[:ASSIGNED_TO]->(Task: Fix)-[:RESOLVES]->(Issue: Server Crash)-[:BLOCKS]->(Project: Alpha)`. (Precise)

Vector Search	~[0.85] "Alice... server crash... fix..."
Graph Traversal	(Alice) → (Fix) → (Server Crash) → (Alpha)
Synthesized	~[0.85] "Alice... server crash... fix..." + (Alice) → (Fix) → (Server Crash) → (Alpha) Factual path + conversational context.

Evaluation Metrics

- **Context Precision/Recall:** `|Relevant ∩ Retrieved| / |Retrieved|`
- **Faithfulness:** % of statements in generated answer directly supported by source context.
- **Answer Relevance:** Cosine similarity between query embedding and answer embedding.
- **NER F1-Score:** `2 * (Precision * Recall) / (Precision + Recall)`

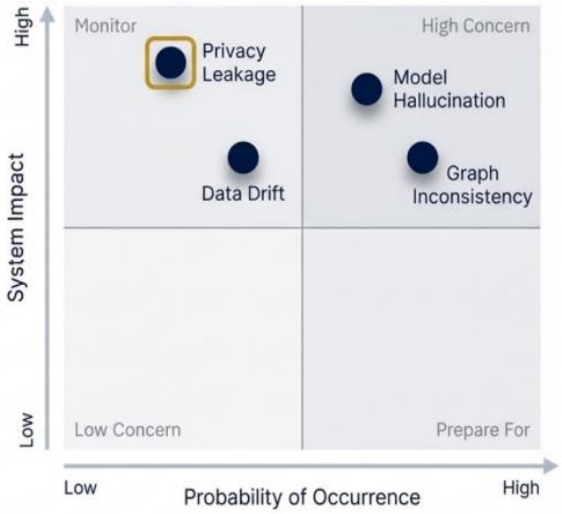
Model Performance Comparison



Technical Risks

- **Model Hallucination:** LLM generates facts not present in the retrieved source context.
- **Data Drift:** Semantic meaning of terms shifts over time, degrading embedding quality.
- **Graph Inconsistency:** Erroneous entity or relation extraction pollutes the knowledge graph.
- **Privacy Leakage:** Sensitive PII is extracted and stored without proper anonymization.

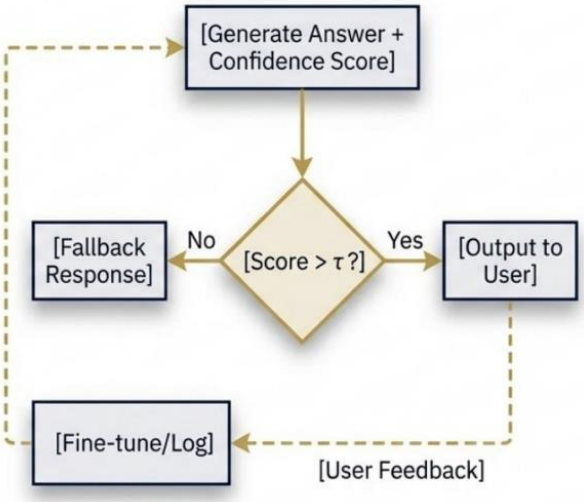
Technical Risk Matrix



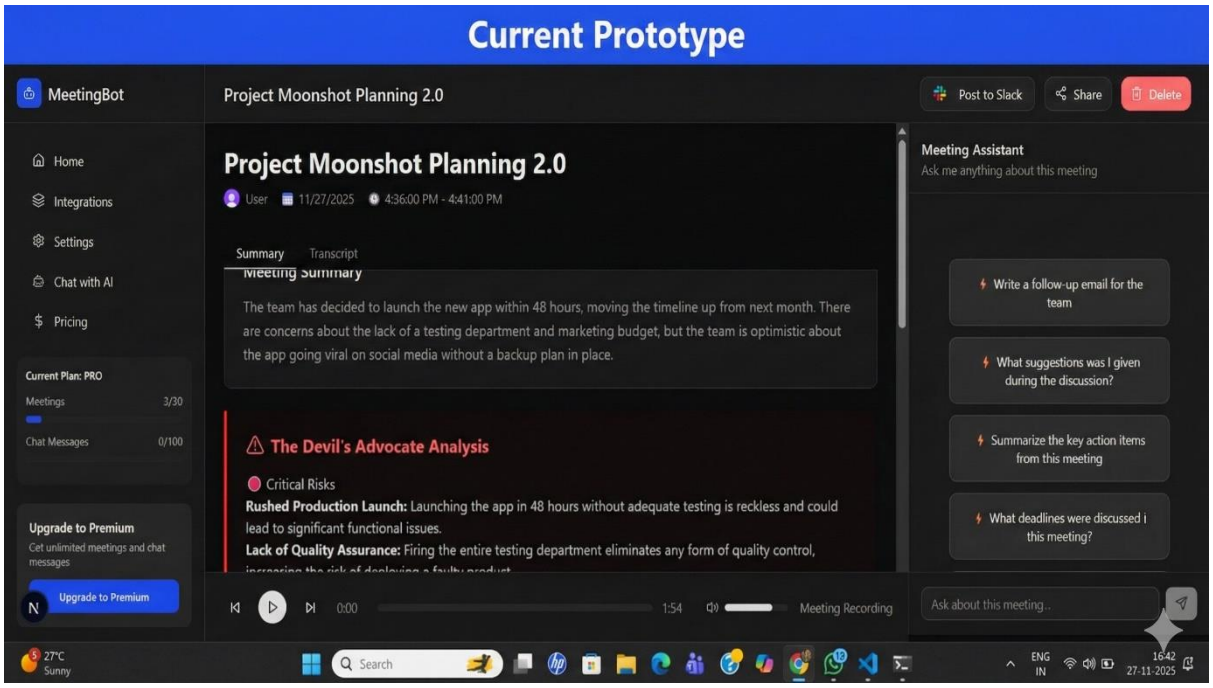
Mitigation Strategies

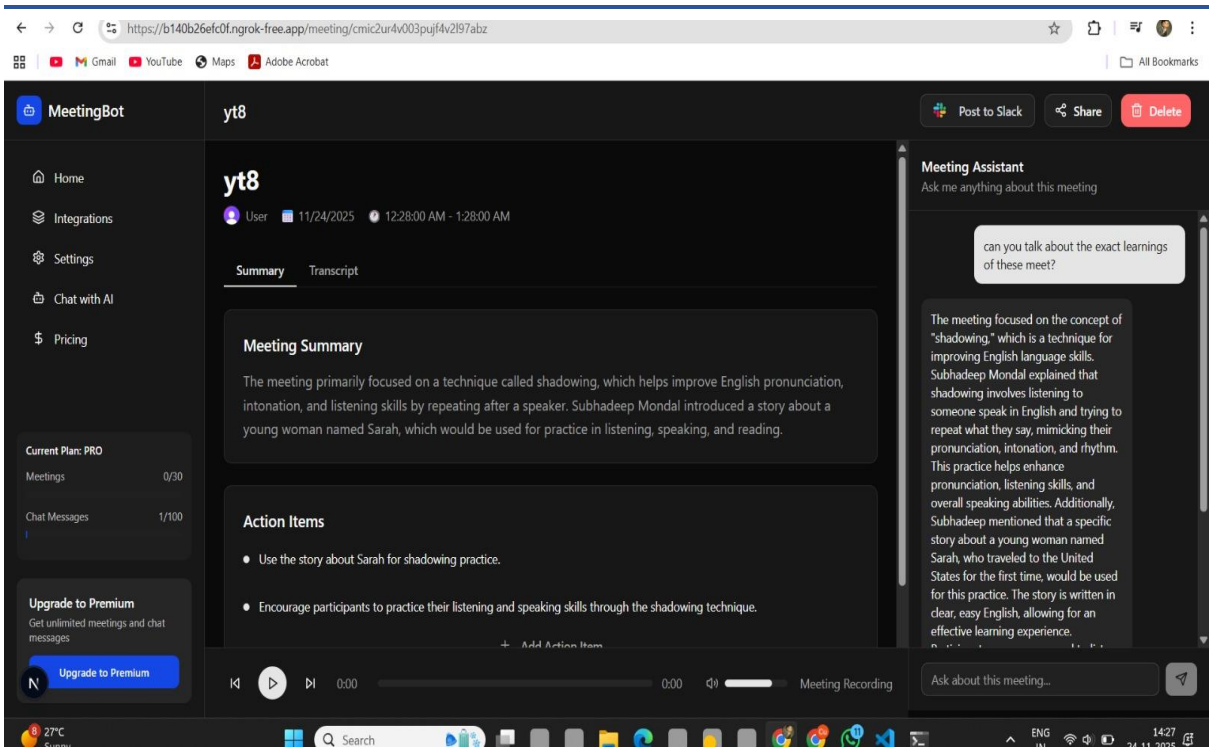
- **Grounding:** Constrain LLM to generate answers *only* from synthesized context.
- **Confidence Scoring:** Surface model confidence scores with every generated answer.
- **Continuous Monitoring:** Track embedding drift and relation extraction accuracy against validation sets.
- **Fallback Logic:** If confidence is below a defined threshold τ, respond with 'Information not available."

Control-Loop Diagram



CURRENT PROTOTYPE





8. Future Improvements

- **On-Device Processing:** Implementing quantized LLMs (e.g., Llama-3-8B) via WebAssembly to process transcripts locally in the browser, enhancing data privacy by keeping sensitive text off the cloud.
- **Voice Biometrics:** Integrating speaker recognition models to automatically tag known users (e.g., identifying "Tarif" vs. "Guest") across different meetings without manual labeling.
- **Predictive Analytics:** Using the Knowledge Graph to predict project delays based on historical risk patterns detected across multiple meeting series.

9. Conclusion

MeetingBot successfully demonstrates that the convergence of **Graph Intelligence** and **Generative AI** offers a robust solution to the problem of "Corporate Amnesia." By architecting a secure, multi-tenant, and event-driven system, this project moves beyond simple transcription to create a tool that actively "reasons" about business data. The implementation proves that AI can function not merely as a passive scribe, but as a strategic analyst, ensuring accountability and preserving institutional knowledge in the modern enterprise.

10. References

1. Vaswani, A., et al. (2017). "Attention Is All You Need." *NeurIPS*.
2. Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS*.
3. Edge, D., et al. (2024). "GraphRAG: Unlocking LLM Discovery on Narrative Private Data." *Microsoft Research*.
4. Devlin, J., et al. (2018). "BERT: Pre-training of Deep Bidirectional Transformers." *NAACL*.
5. Hutto, C.J., & Gilbert, E. (2014). "VADER: A Parsimonious Rule-based Model for Sentiment Analysis." *ICWSM*.
6. Google AI. (2017). "Speaker Diarization with LSTM." *ICASSP*.
7. Neo4j. (2023). "The Property Graph Database Model." *Technical Whitepaper*.
8. Various. (2022). "Serverless Architectures: Throughput and Response Time." *Cloud Computing Journals*.
9. Microsoft. (2023). "Ethical Risks of AI in Workplace Surveillance." *AI Ethics Board*.